

INDICE

Sommario

INDICE.....	2
VISIBILITA' DEI CAMPI E DEI METODI.....	2
STATICO E DINAMICO.....	2
ATTRIBUTI E COMPORTAMENTI DI UNA CLASSE.....	3
Tabella tipi primitivi.....	4
METODO O COMPORTAMENTO.....	4
PARAMETRI.....	5
EREDITARIETA'-> Estendere una sottoclasse da una superclasse.....	5
I COSTRUTTORI.....	6
OVERLOADING → polimorfismo a compiletime.....	6
Scelta del compilatore del metodo da richiamare.....	6
Potrebbe capitare che il compilatore non trovi:.....	6

APPUNTI OCAJP

OVERRIDE → polimorfismo a runtime.....	7
Casi in cui non c'è Override.....	7
UpCast.....	8
DownCast.....	8
UML DIAGRAM.....	9
CICLO.....	9
ARRAY MULTIDIMENSIONALI.....	10
Throwable.....	10
Exception.....	10
ERROR.....	11

VISIBILITA' DEI CAMPI E DEI METODI

Private : Visibile solo all'interno della stessa classe

Default : Visibile all'interno del package

Protected : Visibile in sottoclassi anche in differenti package

Public : Visibile a tutti

STATICO E DINAMICO

Una volta stabilita la visibilità di un campo o metodo è importante capire se un campo o metodo è condiviso fra tutte le istanze oppure è specifico per una istanza.

- Un campo/metodo che non è legato all'istanza di un oggetto, è detto statico; il suo valore è testato a compile time ed è condiviso fra tutte le istanze; può essere modificato e visualizzato in qualsiasi classe sempre se è visibile. La modifica di un attributo statico risulterà in tutto il codice.
- Un campo/metodo dinamico, invece, è strettamente legato al concetto di istanza; ogni oggetto avrà un suo valore per quell'attributo, acquisito solo in fase di esecuzione del programma (a runtime), non avremo un unico valore uguale per tutti ma più valori, uno per ogni istanza di quel tipo di classe.

Il principale compito del costruttore è quello di valorizzare questi attributi dando così ad un oggetto una propria identità. (Oggetto Persona di nome Pippo)

- consideriamo la classe Pippo con due attributi, uno statico ed uno dinamico

Class Pippo

Static int i=2;

Int x=2;

Pippo a=New Pippo();

Pippo b=New Pippo();

a.i = 2; ma anche b.i =2;

se incremento a.i++; a.i=3; ma anche b.i=3;

se incremento a.x++; a.x=3; b.x resta 2;

Un metodo statico può richiamare metodi ed attributi statici della stessa classe senza specificare la classe prima del punto. Può richiamare attributi e metodi da variabili di istanza ma non può valorizzare il this.

Un metodo dinamico può richiamare sia metodi ed attributi statici che dinamici senza specificare la classe o l'istanza prima del punto; e può utilizzare il this per richiamare campi e metodi della classe in una specifica istanza (this); detta questa istanza.

Quindi; se ci trovassimo nella situazione in cui il campo ha lo stesso nome di una variabile o parametro in un metodo dinamico:

- useremo il suffisso this prima del punto per fare riferimento al campo di questa (this) istanza;
- useremo la classe prima del punto per fare riferimento al campo o metodo statico della classe.

campi
static int x=5;
int y=2;
this.y=2; campo di questa istanza
classe.x, campo della classe

ATTRIBUTI E COMPORTAMENTI DI UNA CLASSE

Una classe è un insieme di attributi e comportamenti (legati tra loro, incapsulati) con i quali possiamo definire come è fatto e cosa fa un'oggetto.

Un attributo è una variabile di istanza di un specifico oggetto; se per esempio consideriamo un'oggetto persona; con attributo nome; il nome è un valore legato ad una sola persona e ci permette di identificare una persona precisa, con quel nome (istanza).

L'attributo è diverso da una variabile di metodo o di blocco:

1. Un variabile di metodo/blocco
 - nasce e muore all'interno del metodo/blocco in cui viene dichiarata;
 - o se è definita in un metodo vale all'interno di tutto il metodo;
 - o se è definita in un blocco (if, switch, for, while, {}) sarà visibile solo nel blocco.
 - Deve essere inizializzata (deve avere un valore di partenza; non ne viene assegnato uno per default);
 - Non occorre per definire una caratteristica dell'oggetto.
2. Un attributo o campo
 - non necessita di essere inizializzato; il compilatore gli assegnerà un valore di default.
 - È legato all'istanza dell'oggetto e identifica una caratteristica propria dell'oggetto.
 - Tuttavia se l'attributo ha lo stesso valore per tutti gli oggetti, definendolo statico, condividerà lo stesso valore in tutte le istanze.

Tutti le variabili referenziate (classi) hanno come valore di default NULL; come ad esempio String.

Tabella tipi primitivi

Data Types					
Type Name	Minimum Value	Maximum Value	Default	Size	Literal
byte	-128	127	0	8-bit +/-	_____
short	-32768	32767	0	16-bit +/-	_____
int	-2147483648	2147483647	0	32-bit +/-	3, 077, 0xBAAC
long	-9223372036854775808	9223372036854775807	0	64-bit +/-	3L
float	-1.40239846e-45	3.40282347e+38	0.0	32-bit IEEE float	3.0F, 3.0E2F
double	-4.94065645841246533e-324	1.79769313486231570e+308	0.0	64-bit IEEE float	3.0, 3.0E2, 3.0e2D
boolean	false	true	false	N/A	true, false
char	\u0000	\uffff	\u0000	16-bit Unicode	'3'

METODO O COMPORTAMENTO

Un comportamento di un oggetto è un metodo legato agli attributi dell'oggetto stesso (incapsulamento); con il quale diciamo cosa un oggetto è in grado di fare. (come lo fa non è visibile (astrazione) – il codice implementativo non è visibile).

Un oggetto, con questi metodi ha la possibilità di interagire con il mondo esterno (interfacce), ad esempio un oggetto auto ha una interfaccia freno che determina con il metodo frena, il comportamento dell'auto alla pressione del pedale del freno).

Tuttavia possiamo avere la necessità di fare alcune operazioni interne ad un oggetto e non accessibili dall'esterno, questi metodi non definiscono il comportamento dell'oggetto e saranno privati ed accessibili solamente all'interno della classe.

PARAMETRI

Gestione della memoria di java :

- stack area della memoria che contiene dati elementari, primitive e puntatori
- heap area della memoria che contiene array e oggetti

Le aree di memoria dello heap sono accessibile mediante puntatori memorizzati nello stack.

In java a differenza di altri linguaggi i puntatori non sono dichiarati esplicitamente, sono implicitamente considerati puntatori tutti i riferimenti da array o ad oggetti.

Il comando new

Una volta dichiarato un oggetto mediante una classe o un array, con il comando new è possibile allocare sullo heap l'oggetto (riserva una zona di memoria nello heap) contenente i registri necessari a memorizzare una copia delle variabili pubbliche e private dichiarate nella classe e restituisce un puntatore nello stack a tale zona di memoria.

Tipi primitivi

I tipi primitivi sono variabili che contengono valori elementari:

Tipo di dati	Descrizione
byte	Numero intero a 8 bit (da -128 a 127).
short	Numero intero a 16 bit (da -32768 a 32767).
int	Numero intero a 32 bit (da -2147483648 a 2147483647).
long	Numero intero a 64 bit (da -9223372036854775808 a 9223372036854775807).
float	Numero in virgola mobile a 32 bit.
double	Numero in virgola mobile a 64 bit.
char	Singolo carattere Unicode a 16 bit.
boolean	Valore logico true o false.

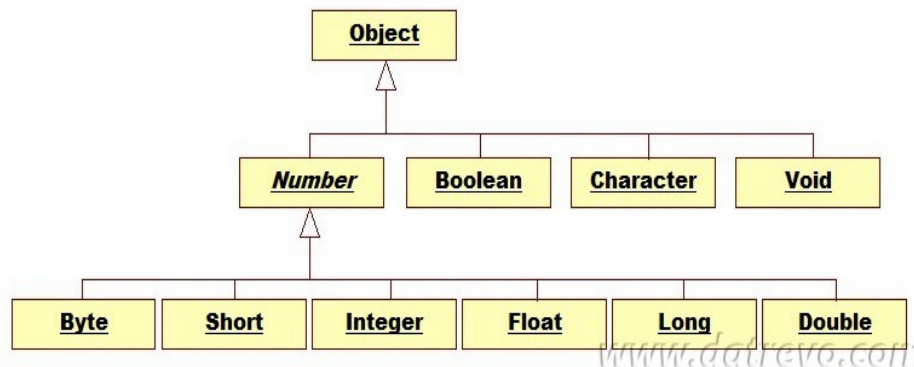
Wrapper Class

Sono Classi che incapsulano i valori primitivi, per ogni tipo avremo la corrispettiva wrapper class:

Tipo primitivo	Classe "wrapper" corrispondente
boolean	Boolean
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

Le classi wrapper sono **immutabili**, e come gli oggetti *String*, in buona sostanza è una classe di tipo final che non può essere ereditata.

L'altra caratteristica fondamentale delle wrapper class derivata dal fatto che sono classi immutabili è che queste classi non hanno la necessità di essere istanziate con keyword new. Tutte le classi wrapper dei tipi primitivi che si riferiscono ad un numero estendono la classe astratta **Number**. Tutte Le altre classi estendono direttamente la superclasse Object.



Passaggio dei parametri

Tutto quello che abbiamo detto fino ad ora ci serve per capire come funziona il passaggio dei parametri in java.

La distinzione del passaggio dei parametri per valore e per riferimento in java non esiste perché quella che viene sempre passata è la copia del valore contenuto nello stack:

- un valore per quanto riguarda primitive, wrapper class e stringhe,
- un riferimento o meglio un puntatore nello heap per quanto riguarda un oggetto

Essendo solamente una copia del valore nel primo caso nel chiamante il valore non cambia:

```
int a=5;
```

```
incrementa(a); // dove il codice sarebbe a++
```

➔ a resta ancora 5 e ciò vale anche per le wrapper class perché sono di tipo final.

Nel secondo caso, invece viene passato il riferimento all'oggetto, puntando nella stessa zona di memoria dello heap, le modifiche fatte saranno persistenti anche nel chiamante.

```
public class Studenti {
    int eta=10; }
```

```

public class Main {
    public static void main(String[] args) {
        Studenti studente=new Studenti();
        nuovoAnno(studente);
        System.out.println(studente.eta);
    }

    static void nuovoAnno(Studenti studente) {
        studente.eta++;
    }
}

```

EREDITARIETA'-> Estendere una sottoclasse da una superclasse

L'ereditarietà è la capacità di una sottoclasse di estendere una superclasse; la sottoclasse eredita tutti i metodi e gli attributi della classe che estende (superclasse o classe padre); a patto che gli stessi siano visibili.

Qualora un metodo non fosse ridefinito nella classe figlia, erediterà dalla classe padre il suo metodo, se non esiste lo erediterà a sua volta da un'altra super classe (fino alla object).

Una volta esaurite le superclassi il metodo verrà ricercato nella classe object; che è la superclasse di tutte le classi. Qualsiasi classe deriva da object o da n superclassi ma che indirettamente derivano comunque tutte da object.

I COSTRUTTORI

I costruttori sono metodi speciali che servono a istanziare una classe; essi creano un oggetto in memoria (Stack e heap) partendo dalla definizione dell'oggetto (classe).

Per costruire un'istanza di una classe figlia, devo necessariamente partire dalla definizione della classe padre; poiché una classe figlia è una classe padre specializzata; siamo obbligati a definire il costruttore di tutte le classi e se non è esplicitamente scritto un costruttore (con o senza parametri); java né assegnerà uno di default senza parametri.

Una classe senza costruttore, implicitamente, avrà un costruttore di default che è richiamato dalla classe figlia come prima istruzione (super()) per ridefinire l'oggetto figlio nel suo costruttore.

Se la classe padre non avesse un costruttore di default ma ne ha uno con parametri; nella classe figlia dovrò esplicitamente richiamare il costruttore della classe padre con parametri con la parola chiave super(parametri); altrimenti java richiamerebbe automaticamente quello di default. (super() implicito)

Il problema nasce proprio in questa circostanza perché io ho già definito un mio costruttore con parametri e potrei aver deciso di non scrivere quello senza parametri e la classe figlia o richiama quello con parametri che ho definito io o devo definire un costruttore senza

parametri vuoto per non avere errori di compilazione in quanto non troverebbe il costruttore di default.

OVERLOADING → polimorfismo a compiletime

Viene scelto il metodo che ha la stessa firma o con l'upcast dei parametri

(early binding -> polimorfismo statico, fatto a compile time)

Nella stessa classe possiamo avere due metodi o due costruttori con lo stesso nome; ma con una lista di parametri differente (firma diversa); questo è l'overload. Il compilatore richiamerà il metodo in overload che ha stessa firma.

La firma del metodo è il nome del metodo con la sua lista di parametri; non sono considerati nella firma né il tipo di ritorno e né le eccezioni lanciate.

Scelta del compilatore del metodo da richiamare

Se per esempio consideriamo il metodo `long somma(4,5)` il compilatore java richiamerà :

1. o il metodo che ha la stessa firma `long Somma(int x, int y)`;
2. oppure il metodo con l'upCast più prossimo dei parametri (`long Somma(int x, long y)`);

Non è un overload il metodo `long Somma(byte x, byte y)`; perché non posso fare un downcast dei parametri, (il metodo somma non è applicabile al tipo di parametri `byte byte` ma a parametri `int int`).

Potrebbe capitare che il compilatore non trovi:

1. nessun metodo (pur facendo l'upcast dei parametri)
2. oppure due metodi hanno la stessa possibilità di essere scelti. (ambiguous)

Un esempio del secondo caso (Ambiguous) potrebbe essere questo:

`Somma(int x, long y);      Somma(long x, int y);`

OVERRIDE → polimorfismo a runtime

Deve avere la stessa firma; il valore ritornato uguale o super Classe del valore ritornato dal metodo in override, l'eccezione o non dichiarata, o sottoclasse dell'eccezione lanciata dal metodo in override

(Late binding -> polimorfismo dinamico, fatto a runtime; tieni in mente l'esempio foo bar)

Una classe figlia può sovrascrivere un **metodo visibile (no private o default ed in package differente)** della classe padre. La nuova classe derivata dalla classe padre, non è uguale alla precedente ma è una specializzazione.

Il legame tra classe figlia e padre ci permette di trattare staticamente una classe figlia come padre e dinamicamente (a runtime) il compilatore richiamerà i metodi in base alla reale classe figlia assegnata.

Questo permette di far ricondurre più classi figlie alla stessa classe padre; dando la possibilità di mettere tutti gli oggetti a fattor comune; esempio :

```
figure f=new Cerchio();
```

```
figure f=new Quadrato();
```

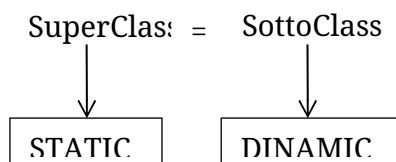
staticamente (compile time) f è una figura e solo **dinamicamente (a runtime)** decideremo che f sarà un cerchio o un quadrato; finché non avvisiamo il compilatore di trattare (cast) f come Cerchio o Quadrato i metodi e gli attributi specifici di Cerchio e Quadrato risulteranno nascosti, gli unici metodi ed attributi che vedremo sono quelli di figura. I metodi in override sono quelli che richiameremo staticamente su figura e dinamicamente sull'oggetto assegnato.

```
figure f=new Cerchio();
```

```
Cerchio c=(Cerchio) f;
```

Per essere certi di sovrascrivere lo stesso metodo della classe padre; dovremo dichiarare nella classe figlia il metodo con la stessa firma.

Polimorfismo a runtime



In Override il tipo statico può essere:

- 1) una classe che estende il tipo dinamico;
`public class SottoClasse extends SuperClasse;`
`superClasse sc=new SottoClasse();`
- 2) un'interfaccia implementata dal tipo dinamico
`interface cancellabile;` `class Matita implements cancellabile;`
`cancellabile matita=new Matita();`

Per l'override tieni in mente l'esempio del calcolo dell'area di una figura, dove `Figura f=new Quadrato(10)`.

In questo esempio abbiamo notato che il metodo `area` richiamato su `f` (`f.area()`) è quello del tipo dinamico, e non `area()` del tipo statico `Figura`.

Il metodo `area()` della classe `Figura` farà solo da segnaposto al metodo dinamico `area()` che sarà realmente chiamato.

Casi in cui non c'è Override

- 1) Caso: errore in compilazione perché ci sono contraddizioni tra il metodo della classe padre e quello della classe figlia.
- 2) Caso: apparentemente c'è override, ma in realtà non c'è.

Caso 1 – Errori in compilazione si hanno quando:

- **La visibilità del metodo in override nella classe figlia è ridotta rispetto al metodo della classe padre.** Esempio metodo classe padre `public`; classe figlia `default`.
- **Il tipo ritornato non è lo stesso oppure non è una sottoclasse del tipo di ritorno dalla classe padre.** Se il metodo `Area` della classe `Padre` ritorna un `Number` allora il metodo `Area` della classe `figlia` può ritornare un `Number` o una sua sottoclasse (esempio `Integer`). In effetti `Number n=new Integer(4)`; ma il contrario non è possibile.

- Il metodo della classe figlia lancia un'eccezione più ampia del metodo nella classe padre. Affinché l'override sia corretto la classe figlia può:
 1. Non lanciare alcuna eccezione
 2. Può lanciare la stessa eccezione
 3. Può lanciare un'eccezione che è sottoclasse dell'eccezione lanciata dal metodo della classe padre.

Caso 2 – Sembra Override ma non lo è

- Il metodo che vogliamo in override nella classe figlia non ha esattamente la stessa firma (nome, tipi e numeri di argomenti identici); ed in tal caso sarebbe richiamato il metodo del tipo statico della classe padre perché ha la firma corretta.
- Ovviamente il metodo con nome differente, facendo attenzione al Key sensitive.

N. B. Si sta parlando di override dei metodi, perciò nel caso ci riferissimo agli attributi, saranno considerati gli attributi della classe statica, ovvero la classe padre (della classe figlia solo con il cast). Esempio `Figura f=new Quadrato();` dove `figura` ha una stringa `tipo="figura"` e `Quadrato` ha una stringa `tipo="Quadrato"`; `f.tipo` è uguale a `"figura"`; sempre se non casto `f` a `Quadrato`; `(Quadrato)f.tipo` è uguale a `"Quadrato"`

UpCast

E' la possibilità di considerare la classe padre al posto di una classe figlia. Nell'overload ad esempio il compilatore cerca un metodo con la stessa firma o con l'upcast dei parametri. Una classe padre puoi considerarla come un contenitore della classe figlia.

DownCast

Non è mai possibile considerare una classe figlia al posto di una classe padre, si avrebbe la perdita di informazioni e ciò non consentirebbe la corretta costruzione dell'oggetto.

Cerchio `c=new Figura();` // non è mai possibile

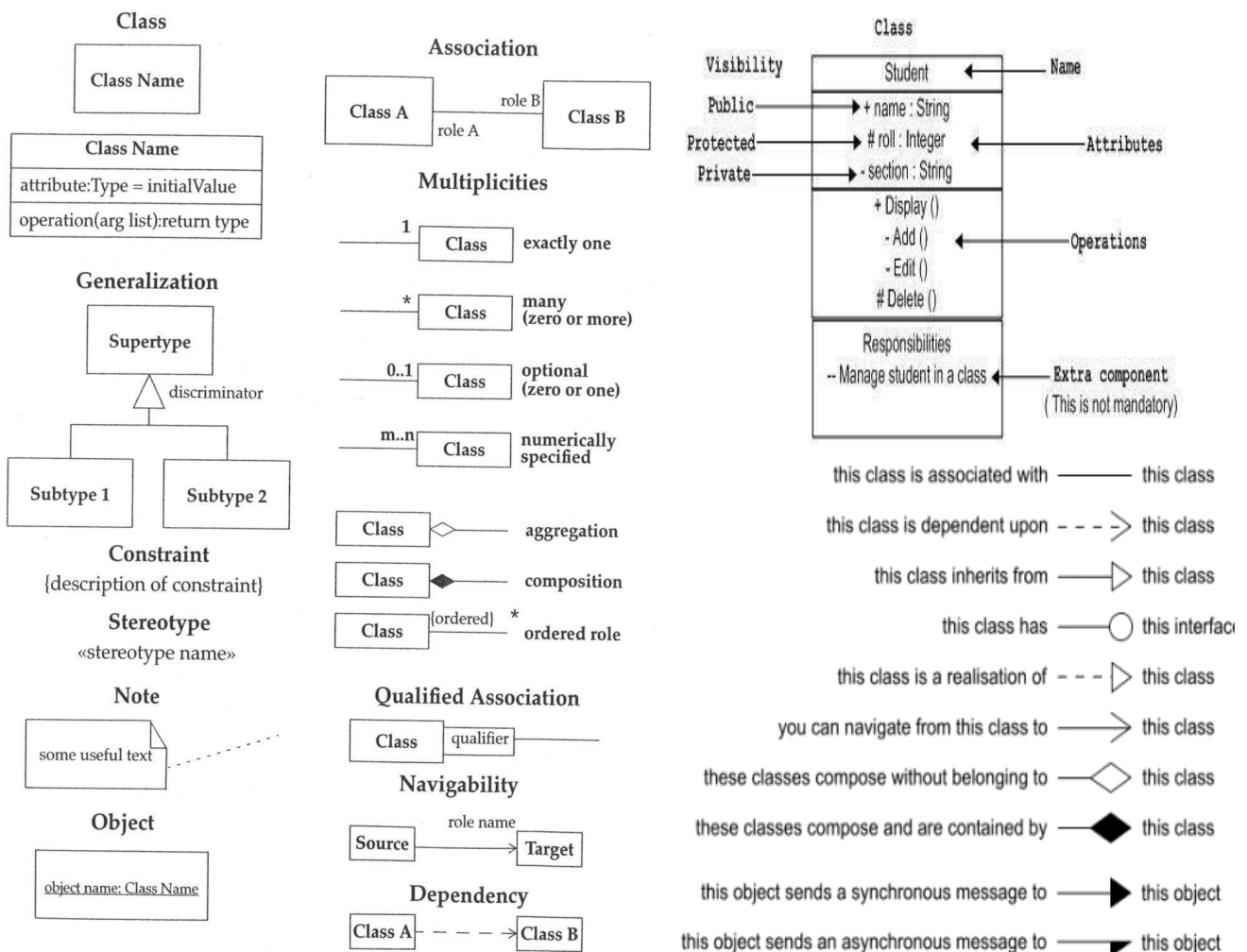
Tuttavia posso ricondurre però un tipo primitivo facendo il downcast anche se è inevitabilmente avrò la perdita della corretta informazione.

`int a=6; Byte b=(Byte) a;`

UML DIAGRAM

Link → <https://dzone.com/articles/uml2-class-diagram-java>

UML diagram



CICLO

Bisogna tenere bene in mente che qualsiasi ciclo (così come l'if) richiede una condizione o un'espressione che ritorni un valore booleano vero o falso. Solo lo switch lavora su interi e caratteri.

Le istruzioni dei cicli vengono eseguite se l'espressione da come valore booleano VERO; mentre si esce dal ciclo; sia while, do...while che for se l'espressione ritorna un valore booleano falso.

esempio:

```
for (int i=1; i<=10; i++)
```

```
    system.out.println(i);
```

```
    i=1; 1<=10 Vero (fai le istruzioni del ciclo)
```

stampa 1;

```
    al termine del ciclo incrementa i=1+1=2
```

```
    2<=10 Vero (fai le istruzioni del ciclo)
```

stampa 2;

fino ad arrivare a $i=11$ ($11 \leq 10$) **Falso** (esci dal ciclo, va all'istruzione successiva alla fine del blocco for)
 Non stampa nulla;
for (;;) infinito

ARRAY MULTIDIMENSIONALI

IL numero di parentesi graffe determina se un array è bidimensioni, tridimensionale, ecc...

Un array bidimensionale è un array di array; un array tridimensionale è un array di un array bidimensionale, ecc...

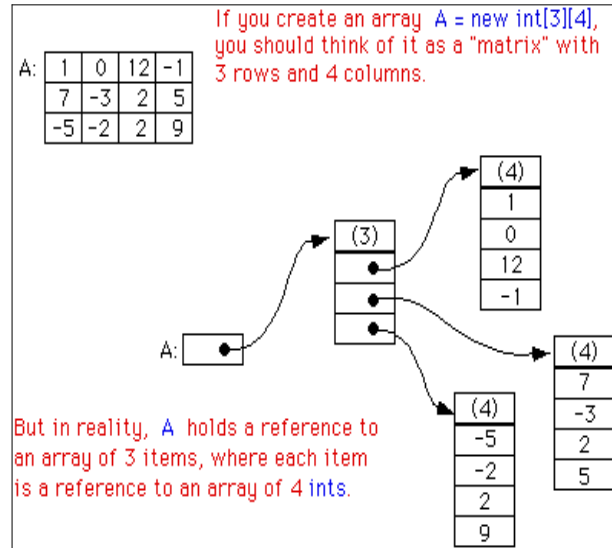
Array

Java Multidimensional Array Example

```
// multidimensional array
int matrix[][] = { {0, 1, 2, 3}, {1, 0, 3, 2}, {2, 3, 0, 1}, {3, 2, 1, 0} };
int row, column;
for ( row = 0; row < 4; row++ ) {
    for ( column = 0; column < 4; column++ )
        System.out.print(" " + matrix[row][column]);
    System.out.println();
}
```

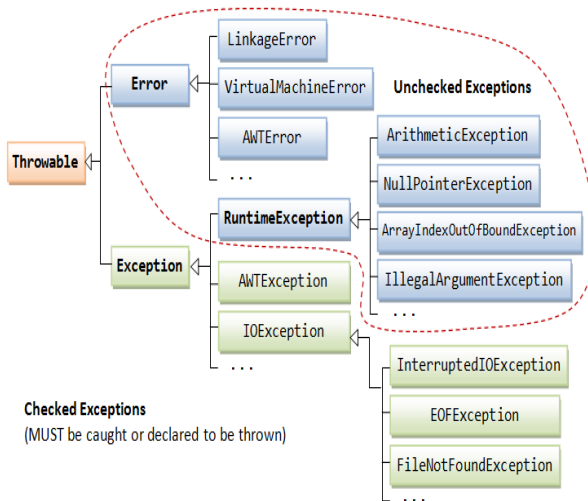
0	1	2	3
1	0	3	2
2	3	0	1
3	2	1	0

© Dr. Chuan R. S. 2000 Structural Programming and Data Structures University of Alberta 19



Throwable

Throwable



Checked: sono eccezione che si presentano a compile time; dovute ad eventi esterni non prevedibili (lettura di un file), perciò siamo obbligati a gestirle (try catch o dichiararle nella firma del metodo con la chiave throws); altrimenti il programma non compila.

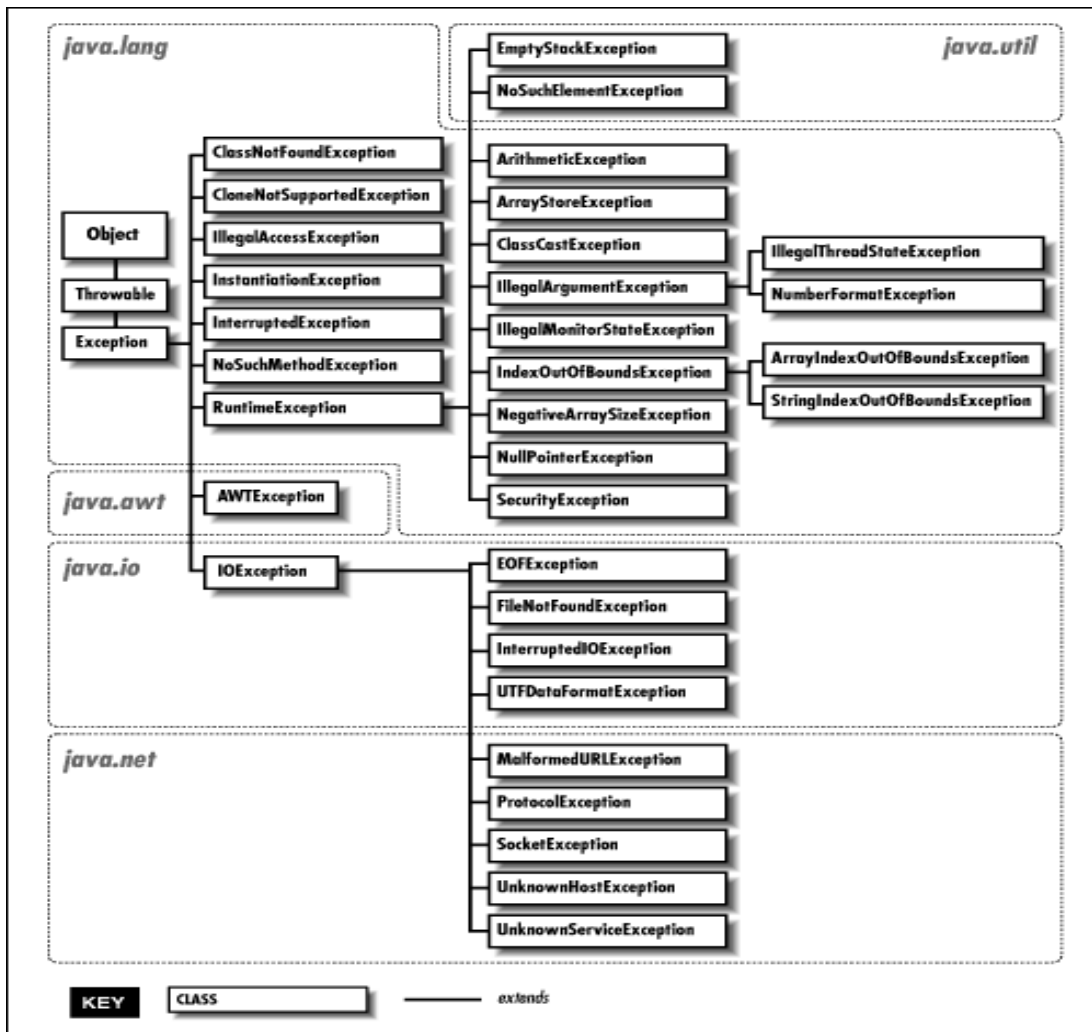
Unchecked: sono eccezioni o errori che si presentano a runtime; più che gestirle andrebbero evitate dato che sono dovute ad un errato modo di programmare.

Non danno problemi in compilazione; l'eccezione genera però un messaggio di errore a Runtime.

Exception

Tenere a mente che le eccezioni di tipo checked vanno gestite e vanno rimandate al chiamante throws o catturate nel try-catch; il metodo che solleva l'eccezione deve rilanciare al chiamante l'eccezione che a sua volta o la cattura o la lancia throws nella firma del metodo.

Eccezioni



ERROR

Sono tutte quelle circostanze in cui inevitabilmente si interrompe l'esecuzione del programma, pertanto è inutile catturare e gestire o lanciare l'eccezione...

Sono considerati Unchecked perché non vanno gestite e si presentano a Runtime.

Error

